

**UNIVERSIDAD MARIANO GÁLVEZ DE
GUATEMALA FACULTAD DE INGENIERÍA EN
SISTEMAS DE INFORMACIÓN Y CIENCIAS DE LA
COMPUTACIÓN.**

Estudiante:

Mauro Jom Lem

Carné:

1966

0902-23-4740

Carrera:

Ingeniería en Sistemas de Información

Curso:

Análisis de Sistemas I

Catedrático:

ING. Edgar Raul Molina Rey



Introducción

El presente documento corresponde a la documentación técnica del proyecto final desarrollado para el curso de Análisis de Sistemas I. El proyecto consiste en la implementación de un sistema automatizado de gestión de inventario para una tienda de tecnología denominada *Tienda Tech*, diseñado y ejecutado completamente en un entorno local con el propósito de optimizar los procesos relacionados con el control de productos, ventas, auditoría y generación de reportes.

La solución fue desarrollada utilizando n8n como plataforma principal de automatización de procesos, PostgreSQL 15 como sistema gestor de base de datos relacional y una interfaz web construida mediante HTML5, CSS3 y JavaScript. Asimismo, se incorporaron mecanismos de notificación automática a través de Telegram y Gmail, permitiendo informar oportunamente sobre eventos relevantes, alertas del sistema y resultados de procesos ejecutados. Toda la infraestructura se encuentra desplegada mediante Docker Compose, garantizando un entorno local reproducible, organizado y libre de dependencias de servicios de pago en la nube.

El desarrollo de este proyecto surge de la necesidad de contar con una herramienta que permita automatizar tareas operativas que tradicionalmente se realizan de forma manual, reduciendo errores, mejorando la trazabilidad de la información y facilitando la administración de los recursos del negocio. A través de la integración de workflows automatizados, persistencia de datos y mecanismos de control, el sistema proporciona una solución centralizada para la gestión eficiente del inventario.

Planteamiento del Problema

La empresa ficticia *Tienda Tech* carece de un sistema automatizado para la gestión de sus operaciones de inventario. Los registros de entradas de mercadería, salidas, ajustes de stock, ventas y actualizaciones de precios se realizan de forma manual, lo que genera diversas dificultades operativas, entre ellas:

- Inconsistencias en el registro del stock disponible debido a errores humanos durante la captura de información.
- Ausencia de alertas automáticas cuando el inventario de un producto desciende por debajo del nivel mínimo establecido.
- Falta de un historial de cambios de precios que permita auditar las modificaciones realizadas al catálogo de productos.
- Limitaciones para generar reportes financieros de manera automática y con información actualizada en tiempo real.
- Escasa trazabilidad de los movimientos de inventario, transacciones de venta y errores operativos.
- Ausencia de mecanismos de notificación para informar eventos críticos, como productos agotados o fallos del sistema.

Ante esta situación, se propone el desarrollo de un sistema de automatización que permita centralizar y optimizar los procesos operativos del negocio, garantizando la integridad de los datos, la trazabilidad de las operaciones y la generación de notificaciones automáticas ante eventos relevantes.

Alcance del Proyecto

El sistema implementado abarca diversos módulos funcionales orientados a la automatización y gestión de los procesos operativos de la empresa.

- El módulo de Inventario permite el registro de productos, la consulta de existencias, el control de entradas y salidas de mercadería, la gestión de mermas y la actualización de precios con historial automático de cambios.
- El módulo de Ventas facilita el registro de ventas con múltiples productos, la validación automática de stock mediante triggers, el cálculo de totales, la consulta del historial de transacciones y la anulación de ventas con restauración automática de existencias.
- El módulo de Clientes incluye el registro y búsqueda de clientes, la consulta de historial de compras y la clasificación automática según la frecuencia de compra, identificándolos como clientes VIP, frecuentes u ocasionales.
- El módulo de Reportes permite generar resúmenes diarios de ventas, consultar los productos más vendidos, analizar ventas por categoría, obtener reportes de ingresos y calcular márgenes de ganancia por producto.
- El módulo de Notificaciones se encarga de emitir alertas por niveles bajos de inventario o productos agotados, así como del envío automático de reportes mediante Telegram y Gmail.
- El módulo de Auditoría registra los movimientos realizados dentro del sistema, mantiene un historial de cambios de precios, almacena las alertas generadas y permite la exportación de información en formato PDF.
- Finalmente, el módulo de Manejo de Errores incorpora un Error Handler global encargado de capturar automáticamente las excepciones producidas en los workflows, registrarlas en la tabla *log_errores* y generar notificaciones inmediatas para facilitar su seguimiento y corrección.

Componentes Utilizados

Componente	Tecnología	Descripción	Puerto
<i>Motor de Automatización</i>	n8n (Docker)	Ejecuta los workflows del sistema y expone los endpoints webhook utilizados por la interfaz web.	5678
<i>Base de Datos</i>	PostgreSQL 15	Almacena la información del sistema e implementa triggers, índices y restricciones de integridad.	5432
<i>Administrador de Base de Datos</i>	Adminer	Proporciona una interfaz web para la administración visual y ejecución de consultas SQL en el entorno local.	8080
<i>Interfaz de Usuario</i>	HTML5, CSS3 y JavaScript	Aplicación web encargada de la interacción con el usuario y de la comunicación con n8n mediante solicitudes HTTP.	3000
<i>Orquestación de Servicios</i>	Docker Compose	Gestiona el despliegue y la comunicación de los contenedores mediante una red interna y variables de entorno.	N/A

Descripción de la Arquitectura

El sistema opera bajo un modelo cliente-servidor desacoplado en un entorno local. La interfaz web actúa como cliente y envía solicitudes HTTP a los endpoints webhook de n8n (puerto 5678). Por su parte, n8n funciona como la capa de lógica de negocio, ejecutando el workflow correspondiente para cada solicitud.

Los workflows interactúan con PostgreSQL para almacenar y consultar información, además de utilizar los nodos de Telegram y Gmail para el envío de notificaciones. PostgreSQL implementa triggers que se ejecutan automáticamente ante operaciones de inserción, actualización o eliminación en las tablas críticas. Estos mecanismos garantizan la integridad del inventario, el cálculo de totales de venta y el registro de cambios de precios sin intervención externa.

Todos los contenedores operan dentro de la red interna *inventario_net* de Docker, evitando que PostgreSQL quede expuesto directamente a Internet. Ante cualquier fallo en un workflow, el Error Trigger de n8n activa el workflow ERR-01, el cual registra el evento en la tabla *log_errores* y envía una notificación inmediata mediante Telegram y correo electrónico.

Diagrama de arquitectura

Modelo de Datos

La base de datos utiliza el esquema *inventario* en PostgreSQL 15. Su estructura está compuesta por 11 tablas relacionales vinculadas mediante claves foráneas, además de 8 triggers automáticos y 10 índices implementados sobre columnas de uso frecuente, con el objetivo de garantizar la integridad de los datos y optimizar el rendimiento de las consultas más utilizadas.

Tablas del Sistema

TABLA	COLUMNAS PRINCIPALES	DESCRIPCIÓN
CATEGORIAS	id, nombre, descripcion, activo	Clasificación de productos por tipo de artículo.
PROVEEDORES	id, nombre, contacto, telefono, email, activo	Empresas o personas encargadas de abastecer el inventario.
CLIENTES	id, nombre, creado_en	Registro de los clientes del sistema.
PRODUCTOS	id, codigo, nombre, categoria_id, precio_venta, stock_actual, stock_minimo, stock_maximo, actualizado_en	Catálogo principal de productos, relacionado con la tabla categorías.
ENTRADAS	id, producto_id, proveedor_id, cantidad, costo_unitario, notas, fecha	Registro de ingresos de mercadería al inventario.
SALIDAS	id, producto_id, cantidad, motivo, notas, fecha	Registro de mermas, ajustes y salidas de inventario.
VENTAS	id, cliente_id, fecha, total	Cabecera de cada transacción de venta realizada.
DETALLE_VENTA	id, venta_id, producto_id, cantidad, precio_unitario	Detalle de los productos incluidos en cada venta.

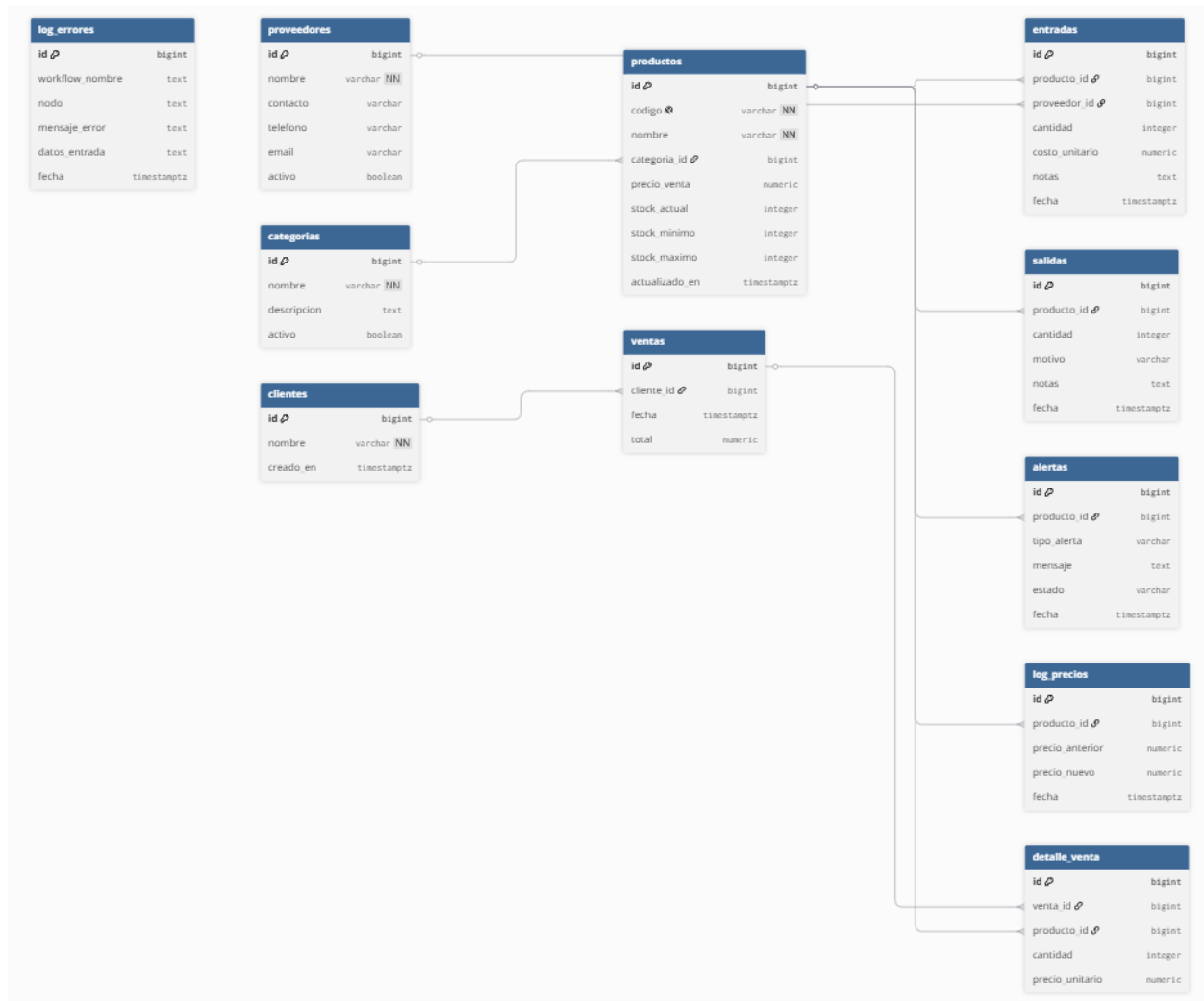
ALERTAS	id, producto_id, tipo_alerta, mensaje, estado, fecha	Registro de alertas por stock bajo o productos agotados.
LOG_PRECIOS	id, producto_id, precio_anterior, precio_nuevo, fecha	Historial de modificaciones realizadas a los precios de los productos.
LOG_ERRORES	id, workflow_nombre, nodo, mensaje_error, datos_entrada, fecha	Bitácora de errores capturados por el Error Handler del sistema.

Triggers Implementados

TRIGGER	EVENTO	ACCIÓN AUTOMÁTICA
FN_UPDATE_TIMESTAMP	UPDATE en productos	Actualiza automáticamente el campo <i>actualizado_en</i> con la fecha y hora de la modificación.
FN_VALIDAR_STOCK_SALIDA	BEFORE INSERT en salidas	Verifica la disponibilidad de stock mediante bloqueo <i>FOR UPDATE</i> y genera una excepción si la cantidad es insuficiente.
FN_STOCK_ENTRADA	AFTER INSERT en entradas	Incrementa el valor de <i>stock_actual</i> según la cantidad registrada en la entrada.
FN_STOCK_SALIDA	AFTER INSERT en salidas	Reduce el valor de <i>stock_actual</i> de acuerdo con la cantidad registrada en la salida.
FN_PROCESAR_DETALLE_VENTA	AFTER INSERT/DELETE en detalle_venta	Valida y descuenta existencias al registrar una venta, y las restablece cuando la venta es anulada.
FN_ACTUALIZAR_TOTAL_VENTA	AFTER INSERT/UPDATE/DELETE en detalle_venta	Recalcula automáticamente el total asociado a la venta.
FN_ALERTAS	AFTER INSERT/UPDATE en productos	Genera alertas de stock bajo cuando las existencias alcanzan el

		mínimo definido y las marca como resueltas cuando se recuperan.
FN_LOG_PRECIOS	AFTER UPDATE en productos	Registra en <i>log_precios</i> los valores anteriores y nuevos cuando se modifica el precio de un producto.

Diagrama entidad-relacion



Descripción de Workflows

El sistema implementa 35 workflows publicados en n8n, distribuidos en siete categorías funcionales: Inventario (6), Ventas (6), Clientes (6), Reportes (9), Notificaciones (5), Auditoría (7) y Manejo de Errores (1). Para efectos de esta documentación, se describen los tres workflows más representativos,

correspondientes a los procesos de ingesta, procesamiento y salida de información.

Workflow de Ingesta: INV-02 Registrar Entrada

Este workflow recibe información desde la interfaz web mediante un Webhook configurado con el método POST en la ruta */registrar-entrada*. Su función es registrar nuevas entradas de mercadería al inventario, actualizar las existencias y generar las notificaciones correspondientes.

Nodos utilizados:

- **Webhook POST /registrar-entrada:** recibe el JSON con *producto_id*, *proveedor_id*, *cantidad*, *costo_unitario* y *notas*.
- **Execute SQL Query (PostgreSQL):** ejecuta la instrucción INSERT en la tabla *entradas*. El trigger *fn_stock_entrada* actualiza automáticamente el stock del producto.
- **Code (JavaScript):** genera el mensaje de notificación utilizando la información del producto y proveedor obtenida mediante una consulta JOIN.
- **Send a text message (Telegram):** envía una notificación al operador con el detalle de la entrada registrada.
- **Send Email (Gmail SMTP):** envía un correo electrónico en formato HTML con la información completa de la operación.
- **Respond to Webhook:** devuelve una respuesta JSON de confirmación a la interfaz web.

Flujo paso a paso:

1. La interfaz web envía una solicitud POST con los datos del formulario de entrada.
2. El Webhook recibe la solicitud y transfiere la información al siguiente nodo.
3. PostgreSQL ejecuta el INSERT correspondiente y el trigger actualiza automáticamente el campo *stock_actual*.
4. El nodo Code genera el mensaje de notificación con los datos del producto y proveedor.
5. Telegram y Gmail envían las notificaciones con el detalle de la entrada registrada.
6. El nodo Respond to Webhook devuelve la confirmación al cliente web.
7. Si ocurre algún error durante el proceso, el Error Trigger activa automáticamente el workflow ERR-01.

Workflow de Procesamiento: VEN-01 Registrar Venta

Este workflow representa el proceso de negocio más complejo del sistema. Recibe un carrito de compra con múltiples productos, valida la disponibilidad de inventario mediante triggers de base de datos, registra la transacción de venta y genera las notificaciones correspondientes.

Nodos utilizados:

- **Webhook POST /registrar-venta:** recibe el *cliente_id* y el arreglo *detalle* con productos, cantidades y precios.
- **Execute SQL Query (PostgreSQL):** ejecuta una CTE (WITH) que registra la venta y su detalle. Los triggers *fn_procesar_detalle_venta* y *fn_actualizar_total_venta* validan existencias, descuentan stock y calculan el total de la venta.
- **Code (JavaScript):** procesa los datos devueltos por la consulta, calcula el total y construye los mensajes para Telegram y el correo electrónico.
- **Send a text message (Telegram):** envía una notificación con el ID de venta, cliente, productos vendidos y total de la transacción.
- **Send Email (Gmail SMTP):** envía un correo electrónico en formato HTML con el detalle completo de la venta.
- **Respond to Webhook:** devuelve la información de la venta registrada a la interfaz web.

Flujo paso a paso:

1. La interfaz web envía una solicitud POST con el carrito de compra del cliente.
2. El Webhook recibe la información y la transfiere al nodo PostgreSQL.
3. La CTE registra la venta y sus detalles. Si algún producto no dispone de existencias suficientes, el trigger genera una excepción y se activa el workflow ERR-01.
4. El nodo Code procesa los datos obtenidos y genera los mensajes de notificación.
5. Telegram y Gmail envían las notificaciones con el resumen de la venta.
6. El nodo Respond to Webhook devuelve la confirmación a la interfaz web.

Workflow de Salida: REP-04 Reporte Mensual

Este workflow representa el proceso de salida automatizada del sistema. Se ejecuta el primer día de cada mes a las 8:00 AM mediante un Schedule Trigger, consulta el resumen de ventas del mes anterior y envía el reporte completo por correo electrónico sin intervención del usuario.

Nodos utilizados:

- ***Schedule Trigger (Cron: 0 8 1 *)**: activa el workflow el primer día de cada mes a las 8:00 AM.
- **Execute SQL Query (PostgreSQL)**: consulta las ventas del mes anterior agrupadas por categoría, incluyendo totales e ingresos generados.
- **Code (JavaScript)**: genera el contenido HTML del reporte y el resumen para Telegram.
- **Send a text message (Telegram)**: envía un resumen mensual con los principales indicadores al operador.
- **Send Email (Gmail SMTP)**: envía el reporte completo en formato HTML.

Flujo paso a paso:

1. El Schedule Trigger ejecuta automáticamente el workflow el día 1 de cada mes a las 8:00 AM.
2. PostgreSQL consulta las ventas registradas durante el mes anterior y las agrupa por categoría.
3. El nodo Code procesa la información obtenida, genera el resumen para Telegram y construye la tabla HTML del correo.
4. Telegram y Gmail envían el reporte mensual a los destinatarios configurados.

Nodos Utilizados en el Sistema

Nodo	Función	Workflows que lo utilizan
Webhook	Punto de entrada para las solicitudes HTTP provenientes de la interfaz web.	Todos los workflows de consulta y registro.
PostgreSQL	Ejecuta consultas y operaciones de lectura y escritura sobre la base de datos.	Todos los workflows que requieren persistencia de datos.
Code (JavaScript)	Procesa, transforma y prepara datos entre los distintos nodos del flujo.	Workflows de notificaciones y reportes.
IF	Realiza el enrutamiento condicional según el resultado de validaciones o consultas.	Workflows de validación de stock y control de datos de entrada.
Schedule Trigger	Ejecuta workflows automáticamente según expresiones cron configuradas.	NOT-01, NOT-02, REP-04, AUD-04 y AUD-05.
Telegram	Envía mensajes mediante la API del bot configurado.	Todos los workflows de notificaciones.
Send Email (Gmail)	Envía correos electrónicos en formato HTML a través de SMTP.	Todos los workflows de notificaciones.

Error Trigger	Captura errores generados en otros workflows para su tratamiento centralizado.	ERR-01 Error Handler Global.
Respond to Webhook	Devuelve la respuesta HTTP correspondiente a la interfaz web.	Todos los workflows que utilizan Webhook.

Manejo de Errores y Bitácora

Error Handler Global: ERR-01

Se implementó un workflow global de manejo de errores denominado **ERR-01 Error Handler Global**. Este workflow utiliza el nodo **Error Trigger** de n8n, el cual se activa automáticamente cuando un workflow configurado presenta un fallo durante su ejecución. El flujo de procesamiento es el siguiente:

1. **Error Trigger:** recibe la información del error, incluyendo el nombre del workflow, el último nodo ejecutado y el mensaje de la excepción.
2. **Code (JavaScript):** extrae y normaliza los campos *workflow_nombre*, *nodo* y *mensaje_error* a partir del objeto JSON recibido.
3. **PostgreSQL:** registra la información del error mediante una instrucción INSERT en la tabla *log_errores*.
4. **Telegram:** envía una notificación inmediata con el nombre del workflow, el nodo involucrado y la descripción del error.
5. **Send Email:** envía un correo electrónico con el detalle completo de la incidencia.

El workflow **ERR-01** se encuentra configurado como *Error Workflow* para los workflows INV-01 al INV-06, VEN-01 al VEN-05, CLI-01 y REP-01 al REP-05. Esta configuración se realiza desde la sección **Settings** de cada workflow dentro de n8n.

Estructura de la Tabla *log_errores*

CAMPO	TIPO DE DATO	DESCRIPCIÓN
ID	BIGINT IDENTITY	Identificador único autoincremental del registro.
WORKFLOW_NOMBRE	TEXT	Nombre del workflow de n8n que generó el error.
NODO	TEXT	Nombre del nodo en el que se produjo la excepción.

MENSAJE_ERROR	TEXT	Descripción del error capturada durante la ejecución del workflow.
DATOS_ENTRADA	TEXT	Información de entrada serializada en formato JSON para facilitar el análisis y diagnóstico del incidente.
FECHA	TIMESTAMP TZ	Fecha y hora del registro, almacenada con la zona horaria de Guatemala.

Otras Bitácoras del Sistema

- **log_precios:** tabla actualizada automáticamente por el trigger *fn_log_precios* cada vez que se modifica el campo *precio_venta*. Almacena el precio anterior, el nuevo valor y la fecha del cambio.
- **alertas:** tabla gestionada por el trigger *fn_alertas*, encargada de registrar alertas de stock bajo. El estado de cada alerta puede cambiar entre *pendiente* y *resuelta* según el nivel de existencias del producto.
- La sección **Auditoría** de la interfaz web permite consultar los registros de *log_errores*, *log_precios* y *alertas*, además de ofrecer opciones de exportación a PDF y envío de notificaciones mediante Telegram y Gmail.

Consideraciones de Seguridad

Variables de Entorno

Todas las credenciales del sistema se gestionan mediante el archivo *.env*, el cual se encuentra excluido del repositorio mediante la configuración de *.gitignore*. El archivo *docker-compose.yml* referencia estas variables utilizando la sintaxis `${VARIABLE}`, evitando la exposición de credenciales en texto plano.

Las credenciales administradas incluyen:

- **POSTGRES_DB, POSTGRES_USER y POSTGRES_PASSWORD:** credenciales de acceso a la base de datos PostgreSQL.
- **N8N_USER y N8N_PASSWORD:** credenciales de autenticación para el acceso a la plataforma n8n.
- **TELEGRAM_BOT_TOKEN:** token de acceso del bot de Telegram generado mediante BotFather.
- **Contraseña de aplicación de Gmail:** configurada en las credenciales internas de n8n para el envío de correos electrónicos.

Autenticación de n8n

El acceso a la interfaz de administración de n8n y a los endpoints webhook se encuentra protegido mediante autenticación básica (*Basic Auth*), habilitada a través de las variables **N8N_BASIC_AUTH_ACTIVE**, **N8N_USER** y **N8N_PASSWORD** configuradas en el contenedor Docker.

Red Interna Docker

Todos los contenedores operan dentro de la red privada *inventario_net*, definida en Docker Compose con el controlador *bridge*. PostgreSQL no se encuentra expuesto directamente a Internet. La comunicación entre n8n y PostgreSQL se realiza mediante el nombre de servicio interno *postgres*, resuelto automáticamente por el sistema DNS de Docker. El puerto 5432 se publica únicamente para acceso administrativo local desde Adminer.

Integridad de Datos

La base de datos implementa integridad referencial mediante claves foráneas en todas las relaciones entre tablas. Adicionalmente, el trigger *fn_validar_stock_salida* utiliza bloqueos *SELECT FOR UPDATE* para prevenir condiciones de carrera en operaciones concurrentes sobre el inventario, garantizando la consistencia de los datos y evitando discrepancias en los niveles de existencias.

Casos de prueba con entradas validas

#	CASO DE PRUEBA	DATOS DE ENTRADA	RESULTADO OBTENIDO
1	Registrar entrada de mercaderia	producto_id: 1, proveedor_id: 1, cantidad: 10, costo_unitario: 500.00	Stock incrementado en 10 unidades. Notificacion enviada por Telegram y Gmail. Registro en tabla entradas.
2	Registrar venta con dos productos	cliente_id: 2, detalle: dos productos con cantidades validas	Venta registrada. Stock descontado por trigger. Total calculado automaticamente. Notificacion enviada.
3	Actualizar precio de producto	Codigo: LAP-001, nuevo precio: Q5200.00 (diferente al actual)	Precio actualizado. Registro automatico en log_precios con precio anterior y nuevo. Notificacion enviada.
4	Registrar salida por merma	producto_id: 3, cantidad: 2, motivo: merma	Stock decrementado. Movimiento visible en modulo de auditoria de la interfaz.
5	Consultar clientes frecuentes	Peticion GET a /clientes-frecuentes (sin parametros)	Lista de clientes con total de compras, gastado y categoria asignada (VIP/Frecuente/Ocasional).
6	Reporte resumen diario	Peticion GET a /resumen-diario	Tabla con detalle de ventas del dia y resumen de ingresos totales, unidades y ticket promedio.
7	Alerta de stock bajo automatica	Stock_actual <= stock_minimo tras una operacion	Alerta creada en tabla alertas por trigger fn_alertas. Notificacion enviada por Cron cada hora.

Casos de prueba con entradas invalidas

#	CASO DE ERROR	DATOS DE ENTRADA INVALIDA	COMPORTAMIENTO DEL SISTEMA
1	Producto inexistente en entrada	producto_id: 9999 (no existe en la base de datos)	Error de integridad referencial en PostgreSQL. ERR-01 registra el fallo

			en log_errores. Notificacion por Telegram y Gmail.
2	Stock insuficiente en salida	cantidad: 999 cuando stock_actual es 5	Trigger fn_validar_stock_salida lanza excepcion. Operacion rechazada. Workflow falla, ERR-01 se activa.
3	Venta con producto sin stock	Producto con stock_actual igual a 0 incluido en el carrito	Trigger fn_procesar_detalle_venta rechaza el INSERT. Venta no registrada. ERR-01 captura y notifica.
4	Proveedor inexistente en entrada	proveedor_id: 9999 (no existe en la base de datos)	Error de clave foranea. Workflow INV-02 falla. ERR-01 registra en log_errores con nombre del workflow y nodo.

Resultados Obtenidos

Todos los casos de prueba con entradas válidas se ejecutaron satisfactoriamente. Los datos fueron almacenados correctamente en la base de datos, los triggers actualizaron el inventario en tiempo real y las notificaciones fueron enviadas mediante Telegram y Gmail dentro de los tiempos esperados.

En los casos de prueba con entradas inválidas, el sistema rechazó las operaciones de forma adecuada. El workflow **ERR-01** registró cada incidente en la tabla *log_errores*, incluyendo el nombre del workflow, el nodo donde se produjo el error y la fecha de ocurrencia. Asimismo, las notificaciones de alerta fueron enviadas de manera inmediata en todos los escenarios evaluados.

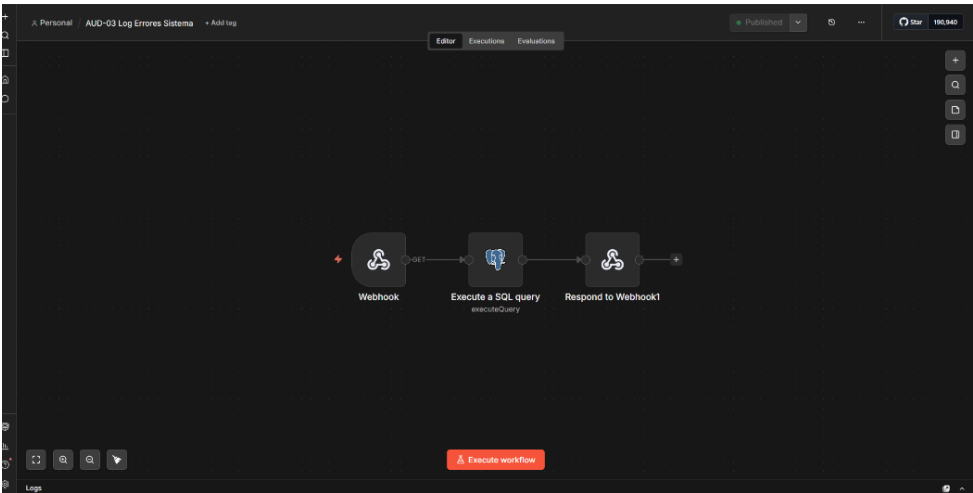
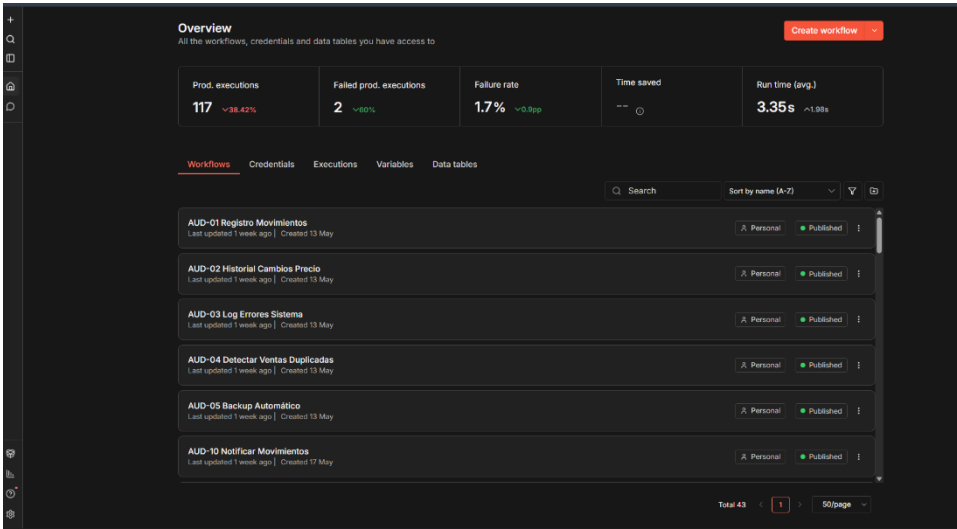
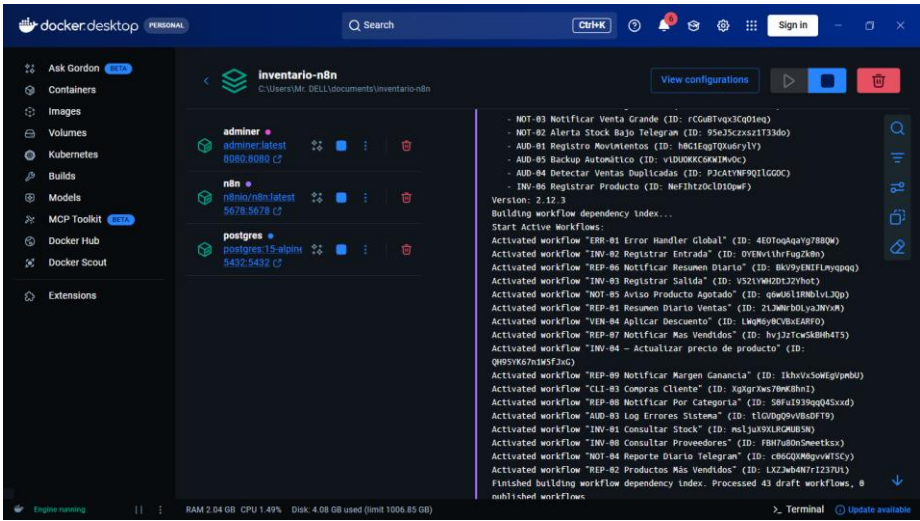
Comando utilizado para generar errores controlados desde PowerShell:

```
Invoke-WebRequest -Uri "http://localhost:5678/webhook/registrar-entrada" `
-Method POST `
-ContentType "application/json" `
-Body '{"producto_id": 9999, "proveedor_id": 9999, "cantidad": 5,
"costo_unitario": 100}'
```

URL del Video Demo:

<https://drive.google.com/file/d/1j0KCWt3brAUw8V2Xs8yYa1okKTE21K-K/view?usp=sharing>

Evidencias de Pruebas



Idioma: Español

Adminer 5.4.2

Login

Motor de base de datos

MySQL / MariaDB

Servidor

posgres

Usuario

Contraseña

Base de datos

Login

Guardar contraseña

BD: inventario db
Esquema: inventario

Modificar esquema

Esquema de base de datos

Procedimientos

Secuencias

Tipos definidos por el usuario

Comando SQL

Importar

Exportar

Crear tabla

registros alertas

registros categorias

registros detalle_venta

registros entradas

registros log_errores

registros log_precios

registros productos

registros proveedores

registros salidas

registros ventas

Tablas y vistas

Buscar datos en tablas (11)

LIKE %%

Condición

Tabla	Motor	Longitud de datos	Longitud de índice	Espacio libre	Incremento automático	Registros	Comentarios
alertas	table	16 384	32 768	?	?	?	?
categorias	table	16 384	32 768	?	?	?	?
clientes	table	16 384	16 384	?	?	?	?
detalle_venta	table	8 192	49 152	?	?	?	?
entradas	table	16 384	49 152	?	?	?	?
log_errores	table	16 384	32 768	?	?	?	?
log_precios	table	8 192	32 768	?	?	?	?
productos	table	16 384	49 152	?	?	?	?
proveedores	table	16 384	16 384	?	?	?	?
salidas	table	16 384	32 768	?	?	?	?
ventas	table	8 192	32 768	?	?	?	?
11 en total	en_US.utf8		155 648	376 832	0		

Selected (0)

Vaciar

Optimizar

Vaciar

Eliminar

Mover a otra base de datos (0)

inventario

Mover

Crear tabla

Crear vista

Procedimientos

Compose

Inbox 116

Starred

Snoozed

Sent

Drafts

Purchases

More

Labels

Search mail

Primary

Promotions 1 item

Social

Updates

Get started with Gmail

Customize your inbox

Change profile image

Import contacts and mail

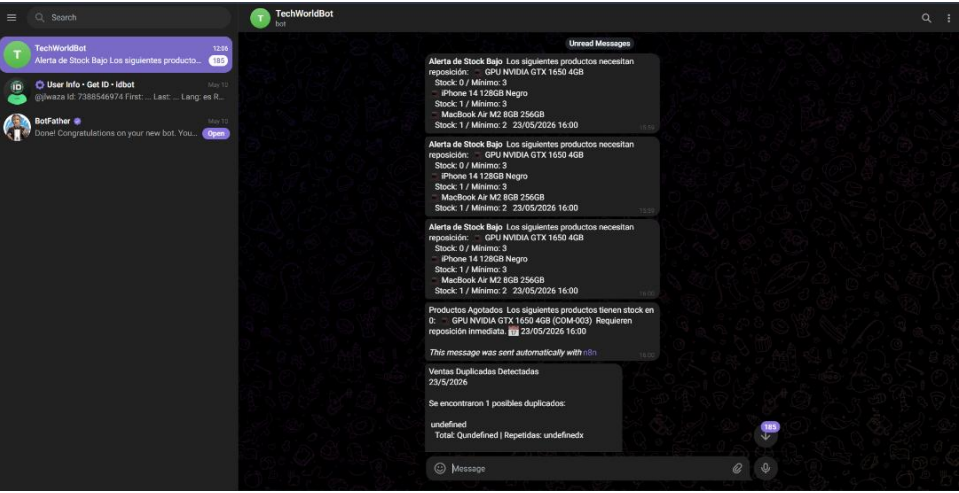
Get Gmail for mobile

me	Ventas duplicadas detectadas - [[\$now.format('dd/MM/yyyy')]] - Ventas Duplicadas Detectadas Total duplicados: 1 Cliente Total Fecha Vencida repetida undefined Undefined undefin...	11:00 AM
me	Ventas duplicadas detectadas - [[\$now.format('dd/MM/yyyy')]] - Ventas Duplicadas Detectadas Total duplicados: 1 Cliente Total Fecha Vencida repetida undefined Undefined undefin...	11:00 AM
me	Alerta de stock bajo - 03/06/2026 - Productos con stock bajo Los siguientes productos requieren reposición: Código Producto Categoría Stock actual Stock mínimo COM-003 GPU NVIDIA...	11:00 AM
me	Alerta de stock bajo - 03/06/2026 - Productos con stock bajo Los siguientes productos requieren reposición: Código Producto Categoría Stock actual Stock mínimo COM-003 GPU NVIDIA...	11:00 AM
me	Alerta de stock bajo - 03/06/2026 - Productos con stock bajo Los siguientes productos requieren reposición: Código Producto Categoría Stock actual Stock mínimo COM-003 GPU NVIDIA...	11:00 AM
me	Alerta de stock bajo - 03/06/2026 - Productos con stock bajo Los siguientes productos requieren reposición: Código Producto Categoría Stock actual Stock mínimo COM-003 GPU NVIDIA...	11:00 AM
me	Alerta de stock bajo - 03/06/2026 - Productos con stock bajo Los siguientes productos requieren reposición: Código Producto Categoría Stock actual Stock mínimo COM-003 GPU NVIDIA...	11:00 AM
me	Ventas duplicadas detectadas - [[\$now.format('dd/MM/yyyy')]] - Ventas Duplicadas Detectadas Total duplicados: 1 Cliente Total Fecha Vencida repetida undefined Undefined undefin...	May 30
me	Alerta de stock bajo - 30/05/2026 - Productos con stock bajo Los siguientes productos requieren reposición: Código Producto Categoría Stock actual Stock mínimo COM-003 GPU NVIDIA...	May 30
me	Alerta de stock bajo - 30/05/2026 - Productos con stock bajo Los siguientes productos requieren reposición: Código Producto Categoría Stock actual Stock mínimo COM-003 GPU NVIDIA...	May 30
me	Alerta de stock bajo - 30/05/2026 - Productos con stock bajo Los siguientes productos requieren reposición: Código Producto Categoría Stock actual Stock mínimo COM-003 GPU NVIDIA...	May 30
me	Ventas duplicadas detectadas - [[\$now.format('dd/MM/yyyy')]] - Ventas Duplicadas Detectadas Total duplicados: 1 Cliente Total Fecha Vencida repetida undefined Undefined undefin...	May 29
me	Alerta de stock bajo - 29/05/2026 - Productos con stock bajo Los siguientes productos requieren reposición: Código Producto Categoría Stock actual Stock mínimo COM-003 GPU NVIDIA...	May 29
me	Alerta de stock bajo - 29/05/2026 - Productos con stock bajo Los siguientes productos requieren reposición: Código Producto Categoría Stock actual Stock mínimo COM-003 GPU NVIDIA...	May 29

Enable desktop notifications for Gmail.

OK

No thanks



Inventario Tienda Tech Dashboard Ver inventario Nuevo producto Registrar entrada Registrar salida Actualizar precio VENTAS Nueva venta Historial ventas CLIENTES Cuentas REPORTES Reportes ACTIVIDAD Auditoria	Sistema de Inventario Tienda Tech último: 3 de junio de 2024 Dashboard Resumen general del negocio TOTAL PRODUCTOS 40 3 días atrás VENTAS DEL DÍA 0 20:00 INVENTARIO PROYECTADO Q0.00 20:00 ALERTAS ACTIVAS 3 Alertas: 1 tipo o 1 agente
--	--

Estado del inventario

Actualizar

Conclusiones

El desarrollo del sistema automatizado de gestión de inventario para Tienda Tech permitió implementar una solución integral capaz de optimizar los procesos relacionados con el control de productos, ventas, clientes, reportes y auditoría. La automatización de estas operaciones contribuye a reducir errores manuales, mejorar la trazabilidad de la información y agilizar la toma de decisiones basada en datos actualizados.

La integración de n8n, PostgreSQL y Docker Compose demostró ser una alternativa eficiente para la construcción de aplicaciones automatizadas en entornos locales. La utilización de workflows, triggers y mecanismos de validación permitió garantizar la consistencia de los datos y la correcta ejecución de los procesos críticos del sistema.

Los resultados obtenidos durante las pruebas evidenciaron el correcto funcionamiento de los módulos implementados, tanto en escenarios normales como ante situaciones de error. El sistema fue capaz de procesar operaciones de manera confiable, mantener actualizados los registros de inventario y generar notificaciones automáticas mediante Telegram y Gmail cuando fue requerido.

Asimismo, la implementación del workflow ERR-01 Error Handler Global fortaleció la capacidad de monitoreo y control de la solución, proporcionando mecanismos de registro, seguimiento y notificación de incidentes que facilitan las tareas de administración y mantenimiento.